

Comentarios Ejercicio SQL

Desarrollo y ayuda visual para entender de mejor manera la asignacion:

<https://learning.lyfter.team/dashboard/duad/roadmap/bases-de-datos/activity/ejercicios-de-sql>

1. Investigue cómo hacer que los PKs se generen automáticamente.

1. Cómo hacer que los PKs se generen automáticamente:

En SQLite se puede hacer que una columna genere automáticamente su número identificador usando:

SQL

INTEGER PRIMARY KEY AUTOINCREMENT

Esto significa que cada vez que se agraga un nuevo registro, SQLite le asigna un número (1, 2, 3, 4...) de forma automática.

En este ejercicio el **“INTEGER PRIMARY KEY AUTOINCREMENT”** se ha utilizado en la tabla **Shopping_Cart** en el campo **cart_id**, porque el carrito no tiene un código natural único.

Ejemplo fácil:

Imaginemos que se estan agregando productos al carrito:

cart_id	buyer_email	product_code
1	juan@gmail.com	P001
2	juan@gmail.com	P003
3	maria@gmail.com	P002

Nosotros **no tendríamos que escribir** el **cart_id** cuando se insertan el resto de los datos. SQLite lo hace por nosotros de forma automatica, lo cual posibilita que el **cart_id** pueda ser utilizado como PK (Primary Key).

2. Utilice los tipos de datos adecuados.

Se han utilizado los tipos de datos más apropiados según el propósito de cada columna, también tomando en cuenta el ejercicio previo:

DATA TYPE (SQLITE)

Columna	Tipo de Dato	Justificación
code, invoice_number	TEXT	Permite códigos alfanuméricos (ej: F00001, P001)
name, brand	TEXT	Texto descriptivo
buyer_email, buyer_phone	TEXT	Correos y números de teléfono
price, total_amount, unit_price, line_amount	REAL	Valores con decimales (precios y montos)
purchase_date	DATETIME	Fecha y hora completa
entry_date	DATE	Solo fecha
quantity, stock_available, cart_id	INTEGER	Números enteros
cashier_code	TEXT	Código del cajero

3. Limitantes de SQLite y cómo se resolvieron

Durante el ejercicio se encontraron las siguientes limitaciones propias de SQLite:

Limitante	Cómo se penso su resolucion
No existe el tipo DECIMAL	Se utilizó REAL (suficiente precisión para este caso)
No soporta VARCHAR(n) con longitud máxima	Se utilizó TEXT (SQLite no restringe el tamaño)
AUTOINCREMENT solo funciona con INTEGER	Se aplicó correctamente solo donde era necesario
Limitaciones al usar ALTER TABLE (no permite eliminar columnas fácilmente ni cambiar tipos)	Se diseñó el esquema inicial correctamente para minimizar cambios
No tiene ENUM ni tipos restrictivos	Se usó TEXT con control en la lógica

A pesar de estas limitaciones, SQLite es ligero, rápido y más que suficiente para este tipo de ejercicios educativos y aplicaciones pequeñas.

Notas adicionales:

- Se respetó el orden solicitado en el ejercicio.
- Se incluyeron claves foráneas ('FOREIGN KEY') para mantener la integridad referencial.
- Se usaron 'NOT NULL' en los campos obligatorios.

TABLA RELACIONAL SQL (VISUAL BOARD)

Tabla Principal	Tabla Relacionada	Tipo de Relación	Campo de Unión	Descripción
Products	Invoice_Items	1 : N	product_code	Un producto puede aparecer en muchas líneas de factura
Invoices	Invoice_Items	1 : N	invoice_number	Una factura puede contener varios productos (líneas)
Products	Shopping_Cart	1 : N	product_code	Un producto puede estar en el carrito de muchos usuarios
Invoices	Buyer (email)	1 : N	buyer_email	Un comprador puede tener múltiples facturas

Relación principal entre Invoices e Invoice_Items

Una factura (**Invoices**) puede contener varios productos. Por eso, la tabla `Invoice_Items` actúa como tabla intermedia. La relación es de **uno a muchos** (1:N). Se relacionan mediante el campo `invoice_number`.

Relación entre Products e Invoice_Items

Un producto (**Products**) puede aparecer en muchas facturas. La relación también es de **uno a muchos** (1:N). Se relacionan mediante el campo `product_code`.

Relación entre Products y Shopping_Cart

Un producto puede estar agregado en el carrito de compras de varios usuarios. Esta es otra relación de **uno a muchos** (1:N), unida por el campo `product_code`.

Relación entre Invoices y los compradores

Un comprador (identificado por `buyer_email`) puede realizar varias compras (facturas). Esto también es una relación de **uno a muchos** (1:N).

Tipo de relación más importante del ejercicio

La relación más compleja es entre la tabla **Invoices** y la tabla **Products**.

Esto es una relación de **muchos a muchos** [N:M] porque:

- Una factura puede tener **varios (muchos) productos**.
- Un producto puede venderse en **varias facturas (Un producto puede estar en muchas facturas.)**

Para resolver esto no podemos conectar directamente las tablas **Invoices** y **Products**. En su lugar, creamos una tabla intermedia llamada **Invoice_Items**.

Esta tabla guarda:

- Qué factura (`invoice_number`)
- Qué producto (`product_code`)
- Cuántos se compraron (`quantity`)
- El precio unitario y el subtotal

Gracias a esta tabla intermedia, podemos saber exactamente qué productos compró cada cliente y en qué facturas aparece cada producto.

¿Cómo funciona Invoice_Items?

Es como el detalle de la compra. Cada fila de esta tabla dice:

“En esta factura (Cliente) se compró este producto, con esta cantidad y este precio”.

Ejemplo:

- Factura **F00001** compró: Laptop y Mouse → Tiene **2 filas** en Invoice_Items
- Factura **F00002** compró: Mouse → Tiene **1 fila** en Invoice_Items
- Factura **F00003** compró: Laptop → Tiene **1 fila** en Invoice_Items

Visualización sencilla (Tabla Intermedia o Tabla de Union [N:M]):

Ejemplo:

- Siendo > P001 una Laptop
- Siendo > P002 un Mouse
- Siendo > P003 un Teclado

invoice_number	product_code	quantity	unit_price	line_amount
F00001	P001	1	1250000	1250000
F00001	P002	1	45000	45000
F00002	P002	1	45000	45000
F00003	P001	1	85000	85000

La tabla **Invoice_Items** es el "puente" entre **Invoices** y **Products**.

Factura (Invoices) → Tabla Intermedia (Invoice_Items)

Factura F00001 → F00001 | P001 (Laptop)

Factura F00001 → F00001 | P002 (Mouse)

Factura F00002 → F00002 | P002 (Mouse)

Factura F00003 → F00003 | P001 (Laptop)

Esto **no se puede guardar** directamente en solo dos tablas. Necesitas una tabla intermedia.

Nota: En este ejemplo se puede ver que nadie compro un teclado (P003) porque no esta presente en ninguna de las facturas del ejemplo.

Integridad de las relaciones

También se definen claves foráneas (**'FOREIGN KEY'**) en las tablas `Invoice\_Items` y `Shopping\_Cart` para garantizar que no se puedan registrar productos o facturas que no existan previamente en las tablas principales.

1. Tabla: Products

Columna	Tipo de Dato	Restricciones	Descripción
code	TEXT	PRIMARY KEY	Código único del producto
name	TEXT	NOT NULL	Nombre del producto
price	REAL	NOT NULL	Precio (decimal)
entry_date	DATE	NOT NULL	Fecha de ingreso
brand	TEXT	-	Marca
stock_available	INTEGER	DEFAULT 0	Stock disponible

Tabla: Shopping_Cart

Columna	Tipo de Dato	Restricciones	Descripción
cart_id	INTEGER	PRIMARY KEY AUTOINCREMENT	ID automático del carrito
buyer_email	TEXT	NOT NULL	Correo del usuario
product_code	TEXT	FK → Products, NOT NULL	Código del producto
quantity	INTEGER	NOT NULL, DEFAULT 1	Cantidad
added_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Fecha agregado

2. Tabla: Invoices

Columna	Tipo de Dato	Restricciones	Descripción
invoice_number	TEXT	PRIMARY KEY	Número de factura
purchase_date	DATETIME	NOT NULL	Fecha y hora de compra
buyer_email	TEXT	NOT NULL	Correo del comprador
buyer_phone	TEXT	-	(Se agregará con ALTER)
cashier_code	TEXT	-	(Se agregará con ALTER) Código del cajero
total_amount	REAL	NOT NULL	Monto total

3. Tabla: Invoice_Items (Productos por factura)

Columna	Tipo de Dato	Restricciones	Descripción
invoice_number	TEXT	FK → Invoices, NOT NULL	Número de factura
product_code	TEXT	FK → Products, NOT NULL	Código del producto
quantity	INTEGER	NOT NULL	Cantidad
unit_price	REAL	NOT NULL	Precio unitario al momento de compra
line_amount	REAL	NOT NULL	Cantidad × Precio unitario